

Tar -Untar for GVF (example) Source: /home/\$USER/zBK20260131/
[keynote 1: the first source was manually compiled to assure development]
[keynote 2: the skeleton is a baby expecting to grow]
[keynote 3: from prototype to maturity, is a GVF metaphor - a clone of the clone] -- [routines]

Establish a target directory path as a library of dated tar files

is to create a directory to store the tar file/s eg xLibs

```
sudo mkdir -p /home/$USER/xLibs
```

GVF is bonded to USB ext4 FS data paths ... (for now)

the archive file name starts with date followed by a letter

indicating last archival eg 20260131a_gvf.tar

a pseudo as a dated-clone [or mkdir -p /path/project/20260207061319SIL]

1. Method I, CREATE portable archive (from INSIDE source)

```
cd /home/$USER/zBK20260131/ # cd source
```

```
sudo tar -czf /home/$USER/xLibs/20260131a_gvf.tar .
```

2. create test fresh-Restore point

```
sudo mkdir -p /home/$USER/zArk
```

3. Restore clone(source.tar) to destination(project-root-path)

```
sudo tar -xzf /home/$USER/xLibs/20260131a_gvf.tar \  
--strip-components=1 -C /home/$USER/zArk
```

The preamble is a preparation that declares what is source to destination: (a) creating the archive somewhere for backup reasons; (b) restore from backup only the skeleton inside the archive.

Steps 1-3 explains how to create an archival point with tar. Once the tar file was created, it can be used to restore. However, the restoration process reflects over a skeleton inside a *project-root-path*. Where `--strip-components` flag affects one element of the original path. As an example:

- original path at the time the archive was created eg zBK20260131 was replaced by 20260131a_gvf due to the cd (change directory inside); -- in this case the restoration of the skeleton has a `project-root-path: 20260131a_gvf` (the name of the tar file);

- since our interest is to manipulate the original `project-root-path`, we used `--strip-components` flag which dictates where the skeleton will be restored. In this case the final destination of the skeleton will be restored per path provided eg (example) `destination=/home/$USER/zArk`.

Method II, is a process that articulates working with full paths which offers more precision.

- tar file name: 20260131b_gvf.tar

- tar full path and file name: /home/\$USER/xLibs/20260131b_gvf.tar

1. create the archive (destination ← source)

```
sudo tar -czf /home/$USER/xLibs/20260131b_gvf.tar -C /home/$USER zBK20260131
```

Note: the accent is on `-C /home/$USER zBK20260131` that keeps the focus on skeleton regardless of its peer/parent.

2. Set a Clean Restore Point

```
sudo rm -r /home/$USER/zArk && sudo mkdir -p /home/$USER/zArk
```

```
ls -l /home/$USER/zArk # should be empty for this experiment
```

3. Restore source.tar to a destination

```
sudo tar -xzf /home/$USER/xLibs/20260131b_gvf.tar \  

```

```
--strip-components=1 -C /home/$USER/zArk
```

Note: the portability used in above example is a direct connection to a skeleton/container which is a backup restore process. The *project-root-path* on the other hand, is a precise definition of the skeleton/container residency.

In scripting/programming the *project-root-path* plays an important aspect to configuration and rules established in architecture compromise, addressing self-check of a skeleton.

Building a foundation,

San Anton [20260131 via AHR “Recovery Point is and should be partial-manual not automated of old vs new, it may include tools outside the `project-root-path`”]

PS: (is hierarchically disconnection from prior AHR-discourse on the matters of GVF – connecting the dots to a larger project)